Coding against interface rather than class

It means writing your code to depend on an interface (a contract) instead of a specific implementation (a class).

- If you depend on an interface, your code is flexible because any class that implements the interface can be used interchangeably.
- If you depend directly on a concrete class, you lock yourself into that specific implementation, making future changes harder.

Example : Instead of saying *"I need a Car"*, you say *"I need something that can drive"*. That way, a Car, a Bike could fit the requirement.

Code against abstraction, not concreteness

This is the broader design principle behind the first idea.

- **Abstraction** means focusing on the essential behavior (what something does).
- **Concreteness** means focusing on the exact implementation (how it does it).

By coding against abstraction, you make your system more flexible, extendable, and testable. You're depending on *what must be done* instead of *how it's done*.

- **Coding against interface** = A practical way to apply coding against abstraction.
- **Coding against abstraction, not concreteness** = The general design principle.